

Fraud Shield AI

A plain-English guide to what we're building, why, and how

Team reference document — read this before the mentor session so every team member can explain the whole system, not just their own piece.

Contents

1. The big picture (in one analogy)
2. What the problem statement is actually asking for
3. Fraud attack vectors — the six ways fraud happens
4. Matching techniques to attack vectors
5. Why a network view catches what a single transaction can't
6. Our architecture: fast lane and slow lane
7. Tech stack — what we're using and why
8. What makes this novel (our differentiators)
9. Build roadmap
10. Reading list

1

The big picture, in one analogy

Imagine a bank branch with a security guard named Sam who has worked there for twenty years. Sam doesn't need a computer to catch fraud — he just knows things:

- He recognizes regular customers and how they normally act.
- If someone who's usually calm rushes in nervous and sweating, he notices.
- If a “customer” claims to be Mr. Sharma but doesn't walk or talk like Mr. Sharma usually does, he's suspicious.
- If five strangers who've never met keep sending money to the same one guy, he notices that pattern too — even though each transaction alone looks fine.
- If he stops someone, he can tell you exactly why — not just “computer says no.”

Fraud Shield AI is a digital version of Sam — built to work across millions of transactions a second, and to never sleep. Every technique in this document is just one of Sam's “senses,” turned into code. Keep this analogy in your head; every section below maps back to it.

2

What the problem statement is actually asking for

Our brief asks for four “senses” and three outcomes. Here is each one translated into plain English, and mapped to Sam.

Brief asks for	Plain English	Sam's equivalent
Transaction behavior	Is this purchase/transfer normal for this person?	“You never spend this much, this fast.”
Device intelligence	Is this the same phone/laptop this person always uses?	“You're using a stranger's phone.”
Geolocation	Does the location make physical sense?	“You were in Delhi 10 minutes ago — now Dubai? Impossible.”
Behavioral biometrics	Does this feel like the same human, based on typing/mouse patterns?	“You're not walking or talking like Mr. Sharma today.”
Prevent fraud	Actually stop the bad transaction, not just flag it later	Sam physically stops the person at the door.

Brief asks for	Plain English	Sam's equivalent
Adapt to new threats	Learn new tricks fraudsters invent, without manual reprogramming	Sam learns a new scam the moment he sees it once.
Explainable	Say why something was flagged, in human words	"I stopped him because his ID photo didn't match his face."

Good news: every one of these seven requirements is covered by the design in this document. Nothing in the brief is left unaddressed.

3

Fraud attack vectors — the six ways fraud happens

Fraud isn't one thing. This is the part most teams skip — and it's the part a mentor will test first. Different attacks need different defenses, so know these six cold.

1. Account takeover (ATO)

What it is: An attacker steals or guesses someone's login (phishing, leaked passwords, SIM swap) and logs in as them.

Think of it like: Like a burglar who has the real house key — the lock isn't broken, but it isn't really the owner walking in.

Signal that reveals it: Sudden change in device, new location, different typing/mouse rhythm, login at an unusual hour.

Why it's hard: The credentials are 100% valid. This isn't about a bad password — it's about catching a bad human behind good credentials.

2. Synthetic identity fraud

What it is: An attacker fabricates a new identity by combining real and fake data (a real ID fragment + a made-up name), then builds up trust over weeks or months before cashing out.

Think of it like: Like slowly building a fake reference and fake resume over a year before finally asking for a huge loan.

Signal that reveals it: Inconsistent biography, reused address/phone/email across “different” people, unusually clean credit history, many accounts created from similar devices.

Why it's hard: No single transaction looks fraudulent. The fraud is in the pattern of construction over time, not any one moment.

3. Transaction / payment fraud

What it is: An attacker uses a stolen card or payment details to make a purchase.

Think of it like: Like someone using your wallet, which they picked from your pocket, to buy things quickly before you notice.

Signal that reveals it: Amount far outside the user's normal range, impossible travel, merchant category mismatch, many transactions in a short window.

Why it's hard: Legitimate users also make occasional large or unusual purchases — false positives here directly annoy real customers.

4. Mule networks / money laundering rings

What it is: Multiple accounts coordinate to move stolen money — funds come in from victims, get split across “mule” accounts, then cashed out.

Think of it like: Like a bucket brigade passing stolen goods hand to hand so no single person is holding very much at once.

Signal that reveals it: Invisible at the single-transaction level. Only visible when you look at the graph: shared devices, shared beneficiaries, circular money flow.

Why it's hard: This is exactly why graph-based detection (Section 5) matters — a model that only looks at one transaction at a time cannot see this pattern, structurally.

5. Bot / automated attacks

What it is: Scripts hammering a login or checkout page — testing thousands of stolen passwords or card numbers automatically.

Think of it like: Like someone trying every key on a giant keyring against your door, as fast as physically possible.

Signal that reveals it: Superhuman request timing, missing normal browser behavior (no mouse movement, no scrolling), same IP/device reused thousands of times.

Why it's hard: Attackers adapt fast — modern bots now mimic human timing, so simple rate-limit rules get bypassed quickly.

6. Adversarial evasion

What it is: Fraudsters who understand how the model scores things deliberately restructure their attack to slip just under the detection threshold (e.g. splitting one big transfer into many small ones).

Think of it like: Like a smuggler who knows the metal detector's exact sensitivity, so they carry just under that amount each time.

Signal that reveals it: Not visible in a single transaction or from a single model. Needs pattern-over-time detection and multiple, independent models that must agree.

Why it's hard: This is the strongest argument for why we use an ensemble instead of one model — no single decision boundary to learn and beat.

4

Matching techniques to attack vectors

This is the single most important slide/table in our whole pitch. It answers the question every mentor asks: “why do you need so many models — why not just one big AI?” The honest answer: each layer exists because it catches something the others structurally cannot see.

Attack vector	Best-fit technique	Why this technique specifically
Account takeover	Behavioral biometrics + device/session anomaly scoring	Credentials are valid; only behavior reveals the impostor
Synthetic identity	Identity graph + anomaly detection at onboarding	Fraud is in the pattern of construction, not one field
Card / transaction fraud	Gradient boosting (XGBoost) + isolation forest	Fast, well-understood on structured data, explains well with SHAP
Mule networks / laundering rings	Graph Neural Network (GNN) on the account-device-transaction graph	Only a relational model sees multi-hop coordination patterns
Bot / automated attacks	Device intelligence + rate/timing anomaly detection	Needs fingerprinting + statistical timing analysis, not deep learning
Adversarial evasion	Ensemble diversity + time-windowed aggregate features	No single decision boundary to game if multiple models must agree

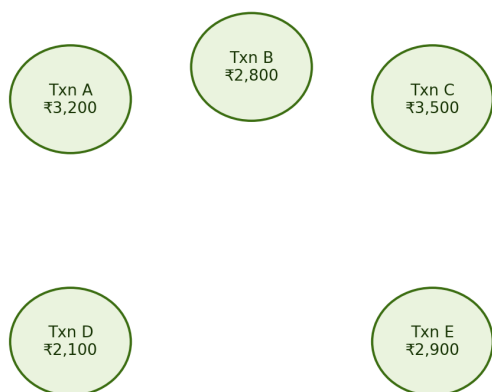
Keep this table in your back pocket during the mentor session — it is the single best answer to almost any “why did you design it this way” question.

5

Why a network view catches what a single transaction can't

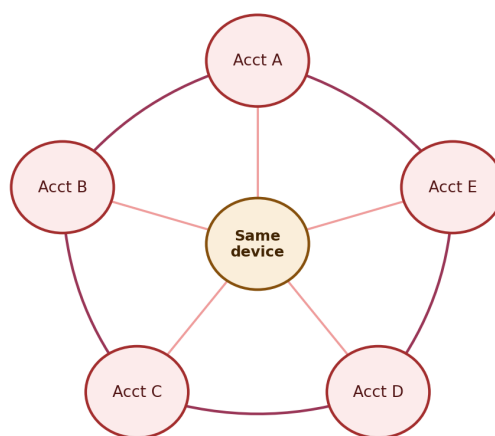
This is the hardest idea to explain, so here's the picture version. Imagine looking at each transaction alone is like looking at one puzzle piece — it just looks like a slightly odd shape, nothing more. But lay all the puzzle pieces on a table and look at how they connect, and suddenly you see a picture: five “different” people who all use the same phone, sending money around in a circle. No single transaction looked wrong. Only the shape of the connections reveals the fraud ring.

Looking at each transaction alone



Each one looks like a normal, small, everyday payment.

Looking at how they connect



Five accounts, one shared device, money moving in a circle: that shape is the fraud.

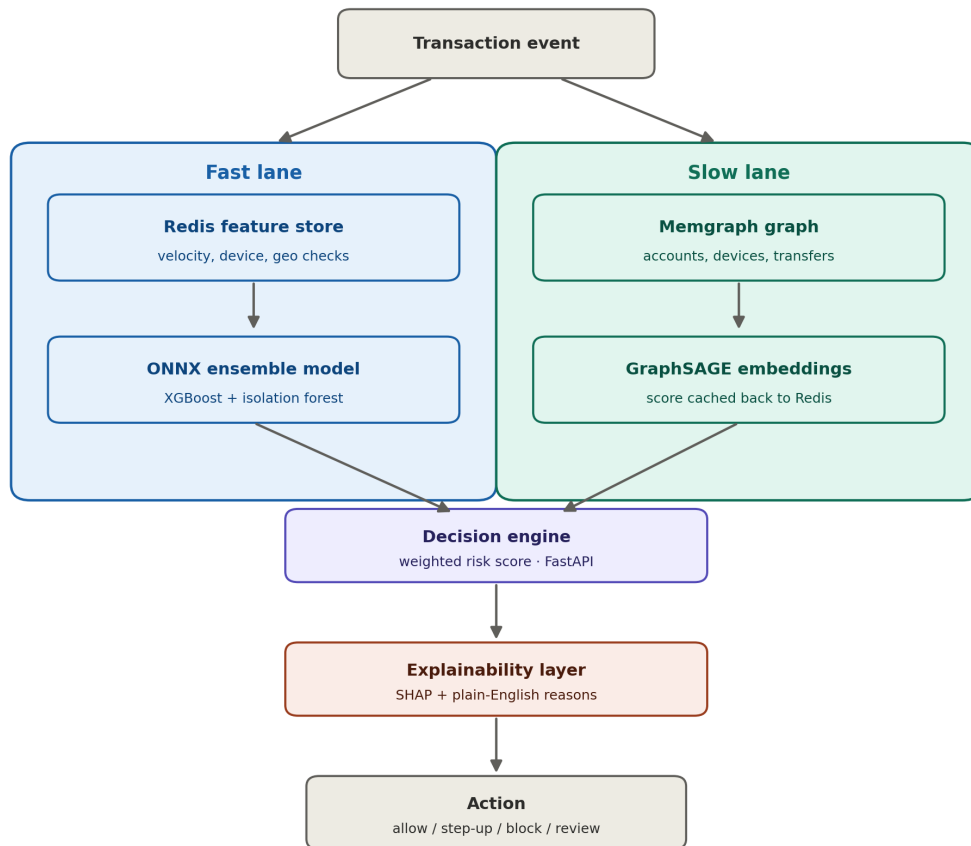
Left: five transactions, each individually unremarkable. Right: the same five transactions, now shown as a network — one shared device, money moving in a closed loop. This relational pattern is what a Graph Neural Network (GNN) is built to detect.

The technique for this is called a **Graph Neural Network**. Instead of asking “does this transaction look weird,” it asks “does this account's neighborhood in the network look weird” — shared devices, shared beneficiaries, unusual connection shapes. This is our single strongest differentiator: most hackathon teams score transactions one at a time and will simply never see this pattern.

6

Our architecture: fast lane and slow lane

Real production fraud systems don't run every model on every transaction in real time — that would be slow and wasteful. The right pattern, and the one we're defending to the mentor, is to split signals into a **fast lane** (must run in milliseconds, on every transaction) and a **slow lane** (can run every few seconds, feeding a cached score into the fast lane).



A transaction event splits into two lanes. The fast lane (blue) does cheap, instant checks. The slow lane (teal) does the expensive relational analysis in the background and caches its answer. Both feed the decision engine, which explains itself before producing a final action.

Fast lane (every transaction, under 10ms)

Redis feature store — keeps rolling stats per account (spend velocity, last known device, last known location) so we never have to query a slow database mid-transaction.

ONNX-served ensemble model — XGBoost/LightGBM + isolation forest, converted to ONNX format for roughly 5–10x faster inference. This is our transaction-level accuracy workhorse.

Slow lane (refreshed every 10–30 seconds)

Memgraph graph database — an in-memory graph built for streaming updates (not static queries like Neo4j), holding accounts, devices, and transfers as a live network.

GraphSAGE (via PyTorch Geometric) — computes a relationship-risk embedding for each account periodically, then caches the score in Redis so the fast lane can look it up instantly. The graph model never sits on the critical path — this is what keeps the system both accurate and fast.

Decision engine and explainability

A single service combines the fast-lane score, the cached graph score, and device/geo checks into one risk tier: **allow / step-up authentication / block / flag for human review**. The combination logic is a transparent weighted formula, not a black-box meta-model — more explainable and easier to defend under questioning. SHAP values are then turned into a plain-English reason, e.g.: “Flagged because: transaction amount is 8x this user’s average, device not seen in 90 days, IP geolocation 4,000km from last login 20 minutes ago.”

Tech stack — what we're using and why

Layer	Choice	Why
Event ingestion	Redpanda (Kafka-API compatible)	Single binary, far easier to run/debug than raw Kafka for a small team; genuinely lower latency
Fast-lane feature store	Redis	Sub-millisecond lookups; the same pattern used in production feature stores (e.g. Feast)
Transaction-level model	XGBoost / LightGBM + Isolation Forest, served via ONNX Runtime	Strong tabular accuracy, explains well with SHAP, and ONNX gives a real, measurable latency win
Relationship / graph layer	Memgraph + GraphSAGE (PyTorch Geometric)	Built for streaming graph updates, not static queries — fits “real-time” requirement directly
Behavioral biometrics	Statistical distance (Mahalanobis / z-score) over session features	Works from very little data per user, stays explainable, and is fast — all three matter more than deep learning here
Explainability	SHAP + template/LLM-assisted narrative	Turns scores into plain-English reasons; template fallback so the demo never breaks on an API timeout
Decision service	FastAPI	Lightweight, fast, easy for the whole team to work in, transparent weighted-scoring logic

Why this combination wins on accuracy, latency, real-world credibility, and buildability: only cheap, fast operations sit on the critical path (Redis lookups, ONNX inference); expensive operations (graph inference, biometric baselining) run in the background and are cached. This mirrors how production fraud platforms are actually architected — it is not a simplified hackathon version, it is the real pattern, just scoped to what three people can build in four weeks.

8

What makes this novel — our differentiators

GNN + ensemble + SHAP is becoming the standard playbook — expect other teams to build some version of it. These are the ideas layered on top that most teams won't have time or the security background to attempt.

Red team vs. Blue team AI (headline differentiator)

Instead of one AI that only detects fraud, we build two that fight each other. A **Red Team AI** automatically invents new fraud patterns (splitting a big transfer into smaller ones, spoofing a device, faking travel patterns) to find gaps in our defense. The **Blue Team AI** (our actual product) has to catch these attempts — and every time it misses one, that failure becomes a training example, so it gets better. This is a real, published approach in both cybersecurity (“AI self-play” red/blue simulations) and fraud research specifically (self-attention GANs for fraud detection). It directly answers “how do you adapt to new threats” with a mechanism you can point to and demo live — not just a claim.

Buildable version for our timeline: a script generates 10–20 adversarial transaction patterns, runs them against the current model, shows what got missed, and retrains on the misses. Demo this as a clear before-and-after, not a fully autonomous live system.

Trust score that behaves like a credit score

Instead of a strict fraud yes/no flag, every account gets a trust score that behaves like temperature: it rises slowly with normal behavior, drops sharply the moment something looks wrong, and slowly returns to normal if nothing else happens. More realistic than a binary flag, and a strong live dashboard visual.

AI-written incident reports

When something is flagged, a small AI writes a short, human-readable case summary from the SHAP output and graph context — the same thing a real fraud analyst would want to read, instead of a bare number.

Canvas/WebGL device fingerprinting

Beyond basic browser user-agent checks, tiny differences in how a device renders graphics (canvas/WebGL fingerprinting) are much harder for a fraudster to fake than a spoofed user-agent string — a good “we thought like an attacker” talking point given our security background.

9

Build roadmap (4 weeks to August 20)

Week	Focus	Key deliverables
Week 1	Foundations	Clean IEEE-CIS / PaySim datasets; build the XGBoost + isolation forest baseline end-to-end; stand up Redpanda event pipeline
Week 2	The differentiator	Build the account/device/transaction graph in Memgraph; train GraphSAGE; get a demo fraud ring visibly detectable
Week 3	Explainability + behavioral layer	Wire up SHAP → plain-English reasons; build biometric scoring; combine all scores into one decision engine
Week 4	Polish, adaptivity, buffer	Build the live demo UI; add the Red vs. Blue adaptivity loop; stress-test; rehearse the pitch; record a backup demo video

If we're behind by week 3: cut the LLM-narrative layer first (a template-based explanation is still legitimate), then consider a pre-recorded biometric demo instead of live capture. Never cut the graph/mule-ring detector or the explainability layer — those are our two biggest differentiators.

Reading list

BRIGHT: Graph Neural Networks in Real-Time Fraud Detection

arXiv:2205.13084 — the reference architecture for real-time GNN fraud scoring.

NVIDIA Technical Blog: Supercharging Fraud Detection with GNNs

Practical, engineering-focused walkthrough of graph-based fraud detection.

Financial fraud detection using graph neural networks: A systematic review

Motie & Raahemi, 2024, Expert Systems with Applications — good citation for the pitch deck.

AI Self-Play — Red Team/Blue Team AI-Driven Simulations

macropraxis.org — the concept behind our adaptivity mechanism.

Self-attention GANs for financial fraud detection

ScienceDirect — the research paper behind our attacker/defender idea, applied specifically to card fraud.

How to Build a Real-Time Fraud Detection System with ClickHouse

oneuptime.com — practical patterns for the analytics layer, if we extend beyond Redis.

Real-Time Fraud Detection in Java with Kafka Streams and Vector Similarity

dev.to — explains the embedded-state, no-database-roundtrip architecture idea in plain terms.

This document is meant to be read once, together, before the mentor session — not memorized line by line. The goal is that any one of us can answer “why did you build it this way” for any part of the system.